UT3 · Práctica de inventario deportivo
Formularios GET y POST y subida de imágenes

Desarrollo Web en Entorno Servidor · 2.º DAW · Curso 2026/2027

Departamento de Informática · IES Ramón Arcas Meca · Lorca

1. Objetivo

Un centro deportivo necesita consultar su material y registrar nuevas fichas. Cada ficha representa un tipo de material y unidades indica cuántos ejemplares hay. Completa la aplicación proporcionada con un formulario GET para consultar el inventario y un formulario POST para crear un objeto Material con una imagen opcional. Las validaciones de los campos ya están implementadas. Conectarás los formularios, mostrarás los resultados y completarás la subida del archivo.

2. Preparar el código de partida

2.1 Archivos proporcionados

Descomprime UT3_Practica_Inventario_deportivo_inicio.zip y abre inventario-deportivo. Los comentarios COMPLETAR 1 a COMPLETAR 7 identifican el trabajo que realizarás en los apartados siguientes.

Archivo o carpeta	Contenido que recibes
composer.json	Namespace App y autocarga PSR-4 de src. También carga las funciones auxiliares con namespace.
src/Modelo y src/Servicio	Clase Material y colección Inventario, con agregar, todos y filtrar terminados.
src/Controlador	CatalogoControlador y AltaControlador coordinan las peticiones. ImagenControlador está terminado.
src/Soporte	formularios.php valida los campos; html.php escapa el HTML. archivos.php recibe el archivo y prepara su destino; falta completar inspeccionarImagen.
public	index.php, alta.php y imagen.php son entradas breves ya terminadas.
vistas	Formularios y listado pendientes. El bloque de errores y la confirmación del alta están terminados.
datos y config	Tres materiales iniciales y bootstrap.php para cargar Composer y preparar el inventario.

2.2 Seguir el recorrido MVC

La carpeta public es la parte accesible desde el navegador. Sus archivos cargan la aplicación y llaman al método mostrar del controlador correspondiente. La recepción de los datos, la validación y la decisión de consultar o crear un material se coordinan en src/Controlador.

Los controladores utilizan Material y Inventario y, al terminar, incluyen una vista para generar el HTML. public/index.php ejecuta CatalogoControlador y muestra vistas/catalogo.php. public/alta.php ejecuta AltaControlador y muestra vistas/alta.php. public/imagen.php ejecuta ImagenControlador para devolver los bytes de la imagen.

Las entradas públicas y la organización MVC ya están preparadas. Completa la lógica marcada en los controladores y las funciones de subida en src/Soporte. Escribe los formularios y la tabla en las vistas.

2.3 Preparar Composer y ejecutar la aplicación

Desde inventario-deportivo, comprueba con composer --version que Composer está disponible y con php -m que están habilitadas mbstring y fileinfo. Si falta alguna extensión, localiza la configuración con php --ini y habilítala. Después ejecuta estas órdenes en la misma carpeta.

```
composer validate
composer dump-autoload
php -S localhost:8000 -t public
```

composer.json ya configura App\ como prefijo PSR-4 asociado a src. composer dump-autoload genera vendor/autoload.php, que carga las clases del proyecto. No tienes que escribir require_once para cada clase.

Las funciones auxiliares pertenecen a App\Soporte y se cargan mediante autoload.files. Los use de clases, funciones y constantes necesarios ya están escritos en los archivos proporcionados. Conserva esos imports al completar el código.

Abre http://localhost:8000/index.php. Conserva la estructura y el código proporcionados. La carpeta storage/imagenes viene creada y debe permitir la escritura de PHP. Al principio solo aparecen los encabezados y enlaces, porque los formularios y el listado están pendientes.

2.4 Mostrar textos con e

La función e está proporcionada en src/Soporte/html.php. Recibe un texto y devuelve sus caracteres especiales escapados mediante htmlspecialchars. Así, <, > y comillas se muestran como texto y no alteran el HTML. Utilízala al mostrar los textos del listado. La forma <?= expresión ?> equivale a imprimir esa expresión con echo. Por ejemplo, esta línea imprime el nombre escapado.

```
<?= e($material->nombre()) ?>
```

3. Consultar los materiales mediante GET

3.1 Construir el formulario

En vistas/catalogo.php, completa el marcador COMPLETAR 1. Crea un formulario con method="get" y action="index.php". Utiliza estos controles y añade una etiqueta label asociada al id de cada uno.

Control	Atributos y opciones
Nombre o fabricante	input de texto con id y name q. Valor inicial vacío.
Categoría	select con id y name categoria. Opciones Todas con valor vacío, Colectivo con colectivo, Raqueta con raqueta y Fitness con fitness.
Buscar	button de tipo submit con el texto Buscar.

Escribe las cuatro opciones de categoría directamente con etiquetas option. El bloque que muestra $errores ya está proporcionado encima del formulario. Conserva ese bloque al añadir los controles.

3.2 Validar y consultar la colección

En src/Controlador/CatalogoControlador.php, completa COMPLETAR 2 dentro de mostrar y antes de incluir la vista. Utiliza validarFiltros, proporcionada en src/Soporte/formularios.php. Recibe el array $_GET y devuelve otro array con valores y errores. valores contiene los textos q y categoria ya recortados. errores relaciona cada campo incorrecto con su mensaje y queda vacío si la consulta es válida.

1. Llama a validarFiltros con $_GET y recoge las claves valores y errores del array devuelto.

2. Guarda el array de la clave valores en $filtros y el de errores en $errores. Este último alimenta el bloque de mensajes proporcionado.

3. Solo si no hay errores, llama a $inventario->filtrar con q y categoria, en ese orden. Guarda el resultado en $materiales.

filtrar está implementado en src/Servicio/Inventario.php. Recibe dos textos, q y categoria, y devuelve un array de objetos Material. Busca q como fragmento del nombre o del fabricante sin distinguir mayúsculas y minúsculas y combina ese resultado con la categoría mediante AND. Un filtro vacío no restringe la consulta.

3.3 Representar el listado

En vistas/catalogo.php, completa COMPLETAR 3 dentro de la condición ya escrita. Si $materiales contiene resultados, recórrelos y crea una tabla con las columnas Nombre, Fabricante, Unidades, Categoría y Fecha de alta. Cada objeto ocupará una fila.

Cada elemento de $materiales es un objeto Material, definido en src/Modelo/Material.php. Sus métodos nombre(), fabricante() y categoria() devuelven texto; unidades() devuelve un entero. fechaAlta() devuelve DateTimeImmutable o null. Muestra Sin fecha si es null. En caso contrario, llama a format("Y-m-d") sobre la fecha para obtener el texto que mostrarás. Utiliza e al imprimir los textos.

Si la colección está vacía, muestra No hay materiales que coincidan. Si los filtros tienen errores, conserva el formulario con sus mensajes y no muestres resultados.

4. Dar de alta un material mediante POST

4.1 Construir el formulario

En vistas/alta.php, completa COMPLETAR 4. Dentro de la rama $creado === null, escribe el formulario con method="post", action="alta.php" y enctype="multipart/form-data".

Campo y control	Regla que ya comprueba el servidor
nombre · input de texto	Obligatorio. Hasta 120 caracteres tras recortar espacios.
fabricante · input de texto	Obligatorio. Hasta 100 caracteres tras recortar espacios.
unidades · input de texto	Entero entre 1 y 3000. Usa texto para poder probar entradas como 12abc.
categoria · select	Opción inicial Elige una categoría con valor vacío. Opciones colectivo, raqueta y fitness, con sus etiquetas en castellano.
fechaAlta · input de texto	Opcional. Fecha posible en formato YYYY-MM-DD. Indica ese formato junto al control.
imagen · input de tipo file	Opcional. Un único archivo. Añade accept="image/jpeg,image/png,image/webp".

Usa el nombre del campo como name e id y asocia un label a cada control. Escribe las opciones de categoría directamente con etiquetas option. Añade el botón Guardar material.

Los controles se escriben como HTML estático y vuelven a su estado inicial después de cada envío. El bloque de errores y la confirmación ya están proporcionados. Conserva ambos al construir el formulario.

4.2 Crear el objeto con los datos validados

En src/Controlador/AltaControlador.php ya están escritas la detección de POST y las llamadas a validarMaterial($_POST) e inspeccionarImagen($_FILES). La primera, definida en src/Soporte/formularios.php, devuelve errores y datos, además de los valores leídos. datos es null si hay errores de campos; con campos válidos contiene los cinco datos convertidos. La segunda devuelve archivo y error. El código reúne los errores y solo entra en el try cuando no queda ninguno.

1. En COMPLETAR 5, dentro del try existente, construye $nuevo como objeto Material usando $datos.

2. Utiliza el constructor de Material, ya importado en el controlador. Recibe nombre, fabricante, unidades, categoria y fechaAlta, en ese orden. Obtén cada argumento de la clave correspondiente de $datos. unidades ya es int y fechaAlta ya es DateTimeImmutable o null.

3. Si no hay imagen ni errores, llama a $inventario->agregar con $nuevo y asigna ese mismo objeto a $creado. agregar recibe un Material, lo incorpora a la colección y no devuelve ningún dato. En el apartado 5 incorporarás el caso con archivo.

La confirmación está proporcionada y mostrará la ficha del objeto y cuatro fichas de material en la colección de ese envío. Conserva los bloques catch ya escritos. No construyas el objeto directamente con $_POST.

5. Completar la subida de la imagen

5.1 Validar tamaño y contenido

En src/Soporte/archivos.php, completa COMPLETAR 6 dentro de inspeccionarImagen. Recibe el array $_FILES. La llamada a recibirImagen, ya escrita en ese archivo, comprueba la estructura y los errores de recepción. Sin archivo, devuelve archivo y error con null. Ante un fallo, archivo es null y error contiene el mensaje. Solo un archivo recibido correctamente llega al código pendiente.

Trabaja con $a, que contiene el archivo recibido. $a["size"] es el tamaño en bytes y $a["tmp_name"] es la ruta temporal. Sustituye el retorno provisional por estas comprobaciones, en este orden.

1. Comprueba que size esté entre 1 y 2 097 152 bytes, ambos incluidos. Si no lo está, devuelve un error y archivo con null.

2. Comprueba el temporal con is_file e is_readable, pasando $a["tmp_name"] a ambas funciones. Devuelven true si la ruta es un archivo y si se puede leer, respectivamente. Si alguna devuelve false, devuelve un error.

3. Crea un objeto finfo con FILEINFO_MIME_TYPE y llama a su método file con la ruta temporal. Estas dos llamadas guardan el MIME detectado en $mime. Si es false o no es una clave de MIME_IMAGENES, rechaza la imagen.

```
$detector = new finfo(FILEINFO_MIME_TYPE);
$mime = $detector->file($a['tmp_name']);
```

4. Para un MIME permitido, añade $a["extension"] con el valor MIME_IMAGENES[$mime]. Devuelve un array con archivo igual a $a y error igual a null. En cualquier rechazo, devuelve archivo con null y error con un mensaje.

MIME_IMAGENES ya relaciona image/jpeg con jpg, image/png con png e image/webp con webp. No uses el nombre original ni el type enviado por el navegador para decidir el formato. Esta función solo inspecciona y no mueve archivos.

5.2 Mover y asociar solo una imagen válida

En src/Controlador/AltaControlador.php, completa COMPLETAR 7 después de construir $nuevo y antes de agregarlo. Solo se llega a este bloque si los campos y el archivo han superado la validación.

1. Si $subida["archivo"] no es null, guarda ese array en $archivo y llama a prepararDestino con $archivo["extension"]. Esta función está proporcionada en src/Soporte/archivos.php.

2. Guarda el resultado en $destino. $destino["nombre"] es el nombre generado con su extensión y $destino["ruta"] es la ruta física de almacenamiento. La función ya comprueba el directorio y evita un destino ocupado. Sus errores se recogen en el catch proporcionado.

3. Llama a move_uploaded_file con $archivo["tmp_name"] como origen y $destino["ruta"] como destino. Devuelve true si mueve la subida HTTP y false si falla. Ante false, guarda un mensaje en $errores["imagen"] y no agregues el material.

4. Si el movimiento funciona, llama a $nuevo->cambiarImagen con $destino["nombre"]. Este método de Material guarda el nombre y no devuelve datos. Después, solo si $errores está vacío, agrega el objeto y asígnalo a $creado.

Sin archivo, el alta continúa con imagen null. Con cualquier error, el bloque proporcionado muestra los mensajes y no aparece una confirmación. La entrada imagen.php, ImagenControlador y la imagen de la confirmación ya están preparados para publicar desde storage/imagenes.

6. Comprobar el resultado

6.1 Búsquedas y alta

El inventario inicial contiene Balón de baloncesto de Cancha Sur, con 24 unidades y categoría colectivo; Raqueta de bádminton de Red Ligera, con 12 unidades y categoría raqueta; y Esterilla de yoga de Movimiento Activo, con 18 unidades y categoría fitness. Solo Esterilla de yoga tiene fecha de alta, 2026-10-24.

Prueba	Resultado esperado
Abrir index.php sin parámetros	Aparecen las tres fichas de material y los filtros vacíos.
Buscar activo y elegir Fitness	Solo aparece Esterilla de yoga. Los controles vuelven a su estado inicial.
Buscar activo y elegir Raqueta	Aparece No hay materiales que coincidan.
Abrir index.php?categoria=otro	Aparece el error de categoría y no se muestran resultados.
Alta válida sin imagen	Se muestra la ficha creada, Sin imagen y cuatro fichas de material en la colección de ese envío.
Alta con unidades igual a 12abc	El bloque proporcionado muestra el error de unidades y no se crea el material.
Alta con fechaAlta igual a 2026-02-30	El bloque proporcionado muestra el error de fecha y no se crea el material.

Para el alta válida puedes utilizar Cono de entrenamiento, Circuito Azul, 30 unidades, categoría colectivo y fecha vacía. Comprueba los mensajes del bloque proporcionado cuando el envío no es válido.

6.2 Archivos y continuidad entre peticiones

Prueba con los demás campos válidos	Resultado esperado
JPEG, PNG o WebP de hasta 2 MiB	Se guarda con nombre generado y se muestra su imagen. Un PNG real llamado imagen.txt se guarda como png.
Texto llamado imagen.jpg	Se rechaza el MIME y no aparece ningún archivo nuevo.
Archivo vacío o mayor de 2 MiB	Se rechaza el tamaño y no aparece ningún archivo nuevo.
Unidades incorrectas y PNG válido	Aparece el error de unidades y el PNG no se mueve.

Cada petición vuelve a cargar las tres fichas de material de datos/materiales.php. El alta correcta muestra el objeto en la respuesta del POST. Al pulsar Volver al catálogo o Nueva alta, comienza otra petición con las tres fichas de material iniciales. La imagen guardada sí permanece en disco.

Comprueba el alta en su propia respuesta. Mantén ese funcionamiento y termina el POST mostrando la confirmación ya preparada.